Guía de Laboratorio 3

Sensores y Librerias

Sistemas Electronicos de Instrumentacion - Sistemas Embebidos 2026

1 Objetivos del Laboratorio

- Comprender y contrastar empíricamente el impacto de las rutinas bloqueantes frente a las no bloqueantes en sistemas embebidos.
- Diseñar e implementar una arquitectura de software basada en un **Super-loop** (planificador cooperativo por tiempo) utilizando el contador de *ticks* del sistema.
- Aplicar buenas prácticas de ingeniería de software integrando bibliotecas de terceros y estructurando módulos propios en lenguaje C que soporten múltiples instancias.
- Adquirir datos del entorno utilizando sensores digitales (HC-SR04), analógicos (LM35) y protocolos de comunicación serie (Acelerómetro SPI).

2 Materiales Requeridos

- Placa de desarrollo **STM32F4Discovery** (STM32F407VG).
- Sensor de distancia ultrasónico **HC-SR04**.
- Sensor de temperatura analógico **LM35** (opcional: dos sensores para probar multi-instancia física).
- Protoboard y cables de conexión (jumpers).
- *Nota:* Para el sensor HC-SR04 se entregará a través del repositorio de la cátedra una biblioteca de código fuente (`hcsr04.h` y `hcsr04.c`) lista para su integración.

3 Desarrollo de los Desafíos

3.1 Desafío 1: HC-SR04, Bibliotecas de Terceros y Actuadores

En la industria, frecuentemente se deben integrar bibliotecas proporcionadas por fabricantes o repositorios open-source. En este primer desafío, utilizará una biblioteca predesarrollada por la cátedra para el sensor de distancia y construirá un sistema de advertencia visual simple.

1. **Inspección del Código:** Abra los archivos `hcsr04.h` y `hcsr04.c` provistos. Analice cómo la biblioteca genera el pulso de *Trigger* y cómo utiliza el periférico de Timer en modo *Input Capture* para medir el tiempo del *Echo*.
2. **Integración:** Incluya la biblioteca en su proyecto e inicialice el sensor utilizando las funciones provistas (ej. `HCSR04_Init()`).

3. **Radar de Proximidad (Interacción visual):** Implemente una lógica en su bucle principal que dispare y lea el sensor, y luego controle los LEDs de la placa según los siguientes rangos de distancia:

   - **Distancia > 50 cm:** Enciende el **LED Verde** (Zona Segura).
   - **Distancia entre 20 cm y 50 cm:** Enciende el **LED Naranja** (Zona de Precaución).
   - **Distancia < 20 cm:** Enciende el **LED Rojo** (Zona de Peligro).

   (Asegúrese de apagar los LEDs correspondientes cuando cambie de zona).

3.2 Desafío 2: LM35, Modularización y Soporte Multi-instancia

A diferencia del desafío anterior, aquí usted deberá abstraer el manejo del sensor LM35 diseñando su propia biblioteca desde cero. Esta biblioteca **no** debe configurar a bajo nivel el hardware (de la inicialización de pines y relojes se encarga el código generado por CubeMX/HAL), sino que debe vincular recursos del microcontrolador a instancias lógicas del sensor, permitiendo manejar varios LM35 simultáneamente.

1. **Estructura y Tipos de Datos:** Cree los archivos `lm35.h` y `lm35.c`. Para permitir la coexistencia de múltiples sensores, defina un tipo de dato estructurado (ej. `typedef struct { ... } LM35_HandleTypeDef;`) que almacene la referencia al periférico ADC (ej. `ADC_HandleTypeDef *hadc`) y el canal (`uint32_t`) correspondiente a cada instancia de sensor.

2. **Fase A - Servicios Bloqueantes:** Su API debe implementar inicialmente las siguientes funciones:

   - `void LM35_Init(LM35_HandleTypeDef *hlm35, ADC_HandleTypeDef *hadc, uint32_t canal)`: Asocia los recursos de hardware (puntero al ADC y canal) a la estructura de la instancia.
   - `float LM35_Read_Blocking(LM35_HandleTypeDef *hlm35)`: Configura dinámicamente el secuenciador del ADC para el canal de esta instancia, dispara la conversión, espera activamente a que termine (`HAL_ADC_PollForConversion`), y retorna la temperatura en grados Celsius.

3. **Fase B - Servicios No Bloqueantes:** Para evitar detener el procesador esperando al ADC, amplíe su API implementando un enfoque no bloqueante (basado en interrupciones del ADC o mediante una máquina de estados / *polling* no bloqueante):

   - `void LM35_Start_Conversion(LM35_HandleTypeDef *hlm35)`: Dispara la lectura en el hardware sin esperar el resultado.
   - `bool LM35_Get_Temperature(LM35_HandleTypeDef *hlm35, float *out_temp)`: Consulta si la conversión ha finalizado. Si hay un nuevo dato válido, lo procesa, lo guarda por referencia en `out_temp` y retorna `true`. Si la conversión aún está en proceso, retorna `false` inmediatamente.

4. **Prueba Unitaria:** Incluya su biblioteca en el `main.c`. Declare e inicialice al menos dos instancias lógicas del sensor (puede usar dos canales físicos distintos, o asignar el mismo canal a ambas para validar la lógica del software) y verifique su correcto funcionamiento utilizando *Live Expressions* (modo Debug).

3.3 Desafío 3: El Acelerómetro y el Problema del Bloqueo

El objetivo de este desafío es evidenciar visualmente cómo las funciones de retardo bloqueantes (como `HAL_Delay()`) destruyen la capacidad de respuesta de un sistema embebido que atiende múltiples tareas.

1. **Lectura Básica:** Configure el SPI para comunicarse con el acelerómetro integrado en la placa. Implemente una lógica que encienda los LEDs (Naranja, Verde, Rojo, Azul) indicando hacia dónde se inclina la placa. Observe la fluidez con la que cambian los LEDs al moverla.

2. **El Problema (Código Bloqueante):** Suponga que necesitamos un indicador de sistema activo (*heartbeat*). Añada a su bucle principal una función que haga parpadear el LED Azul cada 500 ms utilizando la función `HAL_Delay(500)`.

   - *Observe y registre:* ¿Qué ocurre con la respuesta de los demás LEDs cuando usted inclina la placa?

3. **La Solución (Código No Bloqueante):** Elimine el `HAL_Delay()`. Implemente el parpadeo del LED Azul utilizando la función `HAL_GetTick()` para evaluar si han transcurrido 500 ms desde el último cambio de estado, sin detener el flujo principal del programa.

   - *Observe y registre:* Verifique que ahora la placa responde fluidamente a la inclinación mientras el LED Azul parpadea de forma independiente.

3.4 Desafío 4: El Super-loop y la Integración Total

Este es el desafío integrador. Debe combinar el funcionamiento de los tres sensores en un único programa principal utilizando una arquitectura de **Super-loop basado en tiempo** (Time-Triggered Super-loop).

Regla estricta: Queda terminantemente prohibido el uso de la función `HAL_Delay()` u otras demoras por lazos vacíos en cualquier parte del código (excepto para retardos de microsegundos estrictamente necesarios para el hardware, si los hubiera).

Requerimientos del Sistema Integrado:

1. **Tarea 1 (Acelerómetro):** Debe leerse continuamente (en cada iteración del lazo) y actualizar los LEDs direccionales para reflejar la inclinación de la placa.

2. **Tarea 2 (Temperatura):** Utilizando la **Fase B (No Bloqueante)** de su biblioteca `lm35`, el super-loop debe lanzar y recuperar la medición de temperatura de forma concurrente, registrando un nuevo valor consolidado cada **1000 ms** (1 segundo).

3. **Tarea 3 (Distancia):** Utilizando la biblioteca `hcsr04`, se debe lanzar una medición y leer su resultado cada **100 ms**.

4. **Tarea 4 (Alarma Concurrente):** Si el HC-SR04 detecta un obstáculo a menos de 15 cm de distancia, la lógica direccional del acelerómetro para el **LED Rojo** debe ser ignorada, y este LED debe comenzar a parpadear a una frecuencia de **5 Hz** (100 ms encendido, 100 ms apagado). Si el obstáculo se retira, el LED Rojo debe volver a reflejar la inclinación normal.

Sugerencia de Arquitectura (Main Loop):
Utilice variables estáticas o globales para almacenar la última vez que se ejecutó cada tarea temporal (ej. `uint32_t last_time_temp = 0;`) y compárelas contra el *tick* actual (`HAL_GetTick()`) en cada ciclo del `while(1)`.